Module-8 (Advance python programming)

1. Printing on Screen :

Q.1. Introduction to the print() function in Python.

Ans: the print() function is one of the most basic and commonly used functions. It is used to display output on the screen (console). This function helps programmers to show messages, results of calculations, or any kind of data to the user.

Syntax:

print(object(s), sep=' ', end='\n')

Explanation:

- object(s): The data or values you want to display.
- sep (separator): Used to separate multiple objects (default is space).
- end: Defines what should be printed at the end (default is a new line).

Examples:

```
print("Hello World")
print(10)
print("Sum is:", 5 + 3)
```

Output:

```
Hello World
10
Sum is: 8
```

The print() function is very useful for debugging and checking how the program is working step by step.

Q.2. Formatting outputs using f-strings and format().

Ans: Sometimes we need to display output in a structured and readable format. Python provides different ways to format strings, such as f-strings and the format() method.

1. Using f-strings (Formatted String Literals)

F-strings were introduced in Python 3.6 and are considered the most modern and efficient way to format strings.

Syntax:

f"string {variable}"

Example:

```
name = "Mithlesh"
age = 25
print(f"My name is {name} and I am {age} years old.")
```

Output:

My name is Mithlesh and I am 25 years old.

Advantages of f-strings:

- Easy to read and write
- Faster than other methods
- Allows direct use of variables and expressions inside {}

Example with expression:

```
print(f"Sum of 5 and 3 is {5 + 3}")
```

2. Using the format() Method

The format() method is another way to format strings. It replaces placeholders {} with values passed as arguments.

Syntax:

"string {}".format(value)

Example:

```
name = "Mithlesh"
age = 25
print("My name is {} and I am {} years old.".format(name, age))
```

Output:

My name is Mithlesh and I am 25 years old.

Using Indexing:

print("My name is {0} and I am {1} years old.".format(name, age))

Using Named Arguments:

print("My name is {n} and I am {a} years old.".format(n=name, a=age))

2. Reading Data from Keyboard:

Q.1. Using the input() function to read user input from the keyboard.

Ans: Using the input() Function in Python

In Python, the input() function is used to take input from the user through the keyboard. It allows the program to interact with the user by accepting data during execution.

Syntax:

input("message")

Explanation:

- The message (optional) is displayed on the screen to guide the user.
- The function waits for the user to type something and press Enter.
- The entered value is always returned as a string (str).

Basic Example:

```
name = input("Enter your name: ")
print("Hello", name)
```

Output:

```
Enter your name: Mithlesh
Hello Mithlesh
```

Important Point: Input is Always String

No matter what the user enters (number, text, etc.), input() always stores it as a string. If we want to use numbers, we must convert it to the required data type.

Taking Integer Input:

```
age = int(input("Enter your age: "))
print("Your age is:", age)
```

Taking Float Input:

```
price = float(input("Enter the price: "))
print("Price is:", price)
```

Example with Calculation:

```
num1 = int(input("Enter first number: "))
num2 = int(input("Enter second number: "))
sum = num1 + num2
print("Sum is:", sum)
```

Multiple Inputs in One Line:

```
a, b = input("Enter two numbers: ").split()
print("Values are:", a, b)
```

If needed as integers:

```
a, b = map(int, input("Enter two numbers: ").split())
print("Sum is:", a + b)
```

Conclusion

The input() function is very important in Python as it helps to make programs interactive. It allows users to provide data during runtime. However, since it always returns a string, proper type conversion is necessary when working with numbers.

Q.2. Converting user input into different data types (e.g., int, float, etc.).

Ans: Converting User Input into Different Data Types in Python

In Python, when we take input from the user using the input() function, the value is always stored as a **string (str)** by default. However, in many situations, we need to perform calculations or comparisons, which require other data types like **int** or **float**.

To solve this, Python provides **type conversion (type casting)** functions.

Why Type Conversion is Needed

If we do not convert input, Python treats numbers as strings, which can give incorrect results.

Example:

```
num = input("Enter a number: ")
print(num + num)
```

Output:

5 + 5 = 55 (wrong for calculation)

So, we must convert the input into the correct data type.

1. Converting to Integer (int)

Used when we need whole numbers.

Syntax:

```
int(input())
```

Example:

```
num = int(input("Enter a number: "))
print(num + num)
```

Output:

```
Enter a number: 5
10
```

2. Converting to Float (float)

Used when we need decimal values.

Syntax:

float(input())

Example:

```
price = float(input("Enter price: "))
print(price * 2)
```

3. Converting to String (str)

Sometimes we convert other data types into string for display purposes.

Example:

```
num = 10
text = str(num)
print("Number is " + text)
```

4. Multiple Inputs with Type Conversion

We can take multiple values and convert them at the same time using map().

Example:

```
a, b = map(int, input("Enter two numbers: ").split())
print("Sum is:", a + b)
```

5. Other Type Conversions

- bool() → Converts value to True or False
- list(), tuple() → Converts into list or tuple

Example:

```
value = int(input("Enter number: "))
print(bool(value))   # 0 = False, others = True
```

Important Points to Remember

- input() always returns a string

- Use int() for whole numbers
- Use float() for decimal numbers
- Invalid conversion (like text to int) will cause an error

Conclusion

Type conversion is very important in Python when working with user input. It ensures that the data is in the correct format for performing operations. By using functions like int(), float(), and str(), we can easily convert input into the required data type and make our programs more accurate and useful.

3. Opening and Closing Files:

Q.1. Opening files in different modes ('r', 'w', 'a', 'r+', 'w+').

Ans: In Python, files are used to store data permanently. To work with files, we use the open() function. While opening a file, we must specify the **mode**, which tells Python how we want to use the file (read, write, append, etc.).

Syntax:

file_object = open("filename", "mode")

- **filename** → Name of the file
- **mode** → Operation to perform on the file

1. Read Mode ('r')

This mode is used to read the content of a file. It is the default mode.

Features:

- File must already exist
- Only reading is allowed
- Error occurs if file does not exist

Example:

```
file = open("data.txt", "r")
content = file.read()
print(content)
file.close()
```

2. Write Mode ('w')

This mode is used to write data into a file.

Features:

- Creates a new file if it does not exist
- Overwrites existing content
- Only writing is allowed

Example:

```
file = open("data.txt", "w")
file.write("Hello World")
file.close()
```

3. Append Mode ('a')

This mode is used to add new content at the end of the file.

Features:

- Creates file if it does not exist
- Does not delete existing content
- New data is added at the end

Example:

```
file = open("data.txt", "a")
file.write("\nNew line added")
file.close()
```

4. Read and Write Mode ('r+')

This mode allows both reading and writing.

Features:

- File must exist
- Does not overwrite automatically
- Writing starts from the current position

Example:

```
file = open("data.txt", "r+")
print(file.read())
file.write("Added text")
file.close()
```

5. Write and Read Mode ('w+')

This mode allows both writing and reading.

Features:

- Creates file if it does not exist
- Overwrites existing content
- File pointer starts at beginning

Example:

```
file = open("data.txt", "w+")
file.write("Hello Python")
file.seek(0)
print(file.read())
file.close()
```

Important Points

- Always close the file using file.close() after use
- Use seek(0) to move the cursor to the beginning
- Be careful with 'w' and 'w+' because they delete existing data

Conclusion

File modes in Python define how we interact with files. Modes like 'r', 'w', 'a', 'r+', and 'w+' give flexibility to read, write, and update data. Choosing the correct mode is important to avoid data loss and to perform operations correctly.

Q.2. Using the open() function to create and access files.

Ans: In Python, the open() function is used to work with files. It allows us to **create new files**, **read existing files**, and **write or update data** in files. File handling is important when we want to store data permanently instead of keeping it only in memory.

Syntax:

file_object = open("filename", "mode")

Explanation:

- **filename** → Name (or path) of the file
- **mode** → Defines how the file will be used (read, write, append, etc.)
- **file_object** → A variable that represents the opened file

Creating a File

To create a new file, we can use modes like 'w', 'a', or 'x'.

Example using 'w' mode:

```
file = open("data.txt", "w")
file.write("This is a new file.")
file.close()
```

Explanation:

- If the file does not exist, Python will create it
- If it already exists, its content will be overwritten

Accessing (Reading) a File

To read data from a file, we use 'r' mode.

Example:

```
file = open("data.txt", "r")
content = file.read()
print(content)
file.close()
```

Explanation:

- Reads the entire file content
- File must already exist, otherwise it gives an error

Appending Data to a File

To add data without deleting existing content, we use 'a' mode.

Example:

```
file = open("data.txt", "a")
file.write("\nThis is an added line.")
file.close()
```

Using with open() (Best Practice)

Instead of manually closing files, Python provides a better way using with. It automatically closes the file after use.

Example:

```
with open("data.txt", "r") as file:
  content = file.read()
  print(content)
```

Benefits:

- No need to call close()
- Safer and cleaner code

Important Points

- Always choose the correct mode ('r', 'w', 'a', etc.)
- Use with open() for better file handling
- Writing mode ('w') can delete old data

- File path can be relative or absolute

Conclusion

The open () function is essential for file handling in Python. It helps to create, read, and update files easily. By using proper modes and good practices like with open (), we can manage files efficiently and safely.

Q.3. Closing files using close ().

Ans: In Python, after opening a file using the open() function, it is very important to close the file once all operations (reading or writing) are completed. This is done using the close() method.

Closing a file releases the system resources that were being used and ensures that all changes are properly saved.

Syntax:

file_object.close()

Why is close() Important?

- It **frees memory and system resources**
- It ensures that all data is **properly written and saved**
- It prevents **data corruption or loss**
- It avoids unexpected behavior when working with files

Example:

```
file = open("data.txt", "w")
file.write("Hello Python")
file.close()
```

Explanation:

- The file is opened in write mode
- Data is written into the file

- close() is called to save changes and release the file

What Happens if We Don't Close a File?

- Data may not be saved completely
- File may remain locked or busy
- It can lead to memory issues in large programs

Checking if File is Closed

We can check whether a file is closed or not using the closed attribute.

Example:

```
file = open("data.txt", "r")
print(file.closed)   # Output: False

file.close()
print(file.closed)   # Output: True
```

Better Alternative: Using with open()

Instead of manually using close(), Python provides a better way using with. It automatically closes the file after the block is executed.

Example:

```
with open("data.txt", "r") as file:
  content = file.read()
  print(content)
```

Benefit:

- No need to explicitly call close()
- Safer and cleaner code

Conclusion

The close() method is essential in file handling to properly finish file operations. It ensures that data is saved and system resources are released. However, using

with open() is considered a better practice as it automatically handles file closing.

4. Reading and Writing Files :

Q.1. Reading from a file using read(), readline(), readlines().

Ans: In Python, we can read data from a file using different methods depending on our requirement. The most commonly used methods are **read()**, **readline()**, and **readlines()**. These methods help us to access file content in different ways.

1. Using read() Method

The read() method is used to read the entire content of the file at once.

Syntax:

file.read(size)

- **size (optional):** Number of characters to read

Example:

```
with open("data.txt", "r") as file:
  content = file.read()
  print(content)
```

Explanation:

- Reads the full file content
- If size is given, it reads only that many characters

2. Using readline() Method

The readline() method reads one line at a time from the file.

Example:

```
with open("data.txt", "r") as file:
  line1 = file.readline()
```

```
line2 = file.readline()
print(line1)
print(line2)
```

Explanation:

- Each call reads the next line
- Useful when working with large files line by line

3. Using readlines() Method

The readlines() method reads all lines of the file and returns them as a **list**.

Example:

```
with open("data.txt", "r") as file:
  lines = file.readlines()
  print(lines)
```

Output Example:

['Line 1\n', 'Line 2\n', 'Line 3\n']

Explanation:

- Each line is stored as a list item
- Useful when we need to process all lines together

Comparison of Methods

Method	Description
read()	Reads entire file or specified size
readline()	Reads one line at a time
readlines()	Reads all lines and returns a list

Important Points

- Always open file in 'r' (read) mode
- Use with open() for safe file handling

- For large files, prefer readline() to save memory
- readlines() may use more memory for big files

Conclusion

Python provides multiple ways to read files depending on the need. The read() method is useful for small files, readline() is best for reading line by line, and readlines() is useful when working with all lines as a list. Choosing the right method improves performance and efficiency.

Q.2. Writing to a file using write() and writelines().

Ans: In Python, we can store data in files using methods like **write()** and **writelines()**. These methods allow us to save text permanently so that it can be used later.

Before writing, we must open the file in a suitable mode like 'w' (write), 'a' (append), or 'w+'.

1. Using write() Method

The write() method is used to write a **single string** into a file.

Syntax:

file.write(string)

Example:

```
with open("data.txt", "w") as file:
  file.write("Hello Python\n")
  file.write("This is a file handling example.")
```

Explanation:

- Writes text into the file
- If file already exists, 'w' mode will overwrite it
- \n is used to move to the next line

2. Using writelines() Method

The writelines() method is used to write **multiple lines (a list of strings)** into a file.

Syntax:

file.writelines(list_of_strings)

Example:

```
lines = ["Line 1\n", "Line 2\n", "Line 3\n"]

with open("data.txt", "w") as file:
    file.writelines(lines)
```

Explanation:

- Takes a list (or any iterable) of strings
- Does not automatically add newline (\n), so we must include it manually

Difference Between write() and writelines()

Method	Description
write()	Writes a single string
writelines()	Writes multiple strings (list)

Appending Data

To add data without deleting existing content, we use 'a' mode.

Example:

```
with open("data.txt", "a") as file:
    file.write("\nNew content added.")
```

Important Points

- Always open file in correct mode ('w' or 'a')
- write() returns number of characters written
- writelines() does not add new lines automatically
- Use with open() for safe file handling

Conclusion

The write() and writelines() methods are used to store data in files in Python. While write() is used for single strings, writelines() is useful for writing multiple lines at once. Choosing the correct method helps in efficient file handling.

5. Exception Handling:

Q.1. Introduction to exceptions and how to handle them using try, except, and finally.

Ans: In Python, an **exception** is an error that occurs during the execution of a program. When an exception happens, the program stops running and shows an error message.

To prevent the program from crashing, Python provides a way to **handle exceptions** using try, except, and finally blocks.

Common Examples of Exceptions

- **ZeroDivisionError** → Dividing a number by zero
- **ValueError** → Invalid input (e.g., converting text to number)
- **FileNotFoundError** → File does not exist
- **TypeError** → Wrong data type used

1. Using try and except

The try block contains code that may cause an error.
The except block handles the error if it occurs.

Syntax:

```
try:
  # risky code
except ExceptionType:
  # handling code
```

Example:

```
try:
    num = int(input("Enter a number: "))
    print(10 / num)
except ZeroDivisionError:
    print("You cannot divide by zero.")
except ValueError:
    print("Invalid input. Please enter a number.")
```

2. Using Multiple except Blocks

We can handle different types of errors separately.

Example:

```
try:
    a = int(input("Enter first number: "))
    b = int(input("Enter second number: "))
    print(a / b)
except ValueError:
    print("Please enter valid numbers.")
except ZeroDivisionError:
    print("Division by zero is not allowed.")
```

3. Using finally Block

The finally block always executes, whether an exception occurs or not.

Syntax:

```
try:
    # code
except:
    # handle error
finally:
    # always runs
```

Example:

```
try:
    file = open("data.txt", "r")
```

```
    print(file.read())
except FileNotFoundError:
    print("File not found.")
finally:
    print("Execution completed.")
```

4. General Exception Handling

We can also catch all exceptions using a general Exception class.

Example:

```
try:
    x = int(input("Enter number: "))
    print(10 / x)
except Exception as e:
    print("Error occurred:", e)
```

Important Points

- try block contains risky code
- except block handles errors
- finally block always runs
- Proper exception handling makes programs more stable and user-friendly

Conclusion

Exceptions are common in programming, but they can be handled using try, except, and finally. This prevents the program from crashing and ensures smooth execution. Proper exception handling improves the reliability and usability of the program.

Q.2. Understanding multiple exceptions and custom exceptions.

Ans: In Python, errors can occur in different ways during program execution. To handle these properly, we use **multiple exception handling** and sometimes create our own **custom exceptions** for better control.

1. Multiple Exceptions in Python

Sometimes a single block of code can cause different types of errors. In such cases, Python allows us to handle multiple exceptions using multiple except blocks.

Syntax:

```
try:
   # risky code
except ExceptionType1:
   # handle first type of error
except ExceptionType2:
   # handle second type of error
```

Example:

```
try:
   a = int(input("Enter a number: "))
   b = int(input("Enter another number: "))
   print(a / b)
except ValueError:
   print("Invalid input. Please enter numbers only.")
except ZeroDivisionError:
   print("Cannot divide by zero.")
```

Explanation:

- If user enters text → ValueError
- If user enters 0 → ZeroDivisionError
- Each error is handled separately

Handling Multiple Exceptions in One Block

We can also handle multiple exceptions together in a single except block.

Example:

```
try:
   num = int(input("Enter a number: "))
```

```
    print(10 / num)
except (ValueError, ZeroDivisionError):
    print("An error occurred. Please check your input.")
```

2. Custom Exceptions in Python

Custom exceptions are user-defined exceptions. We create them when built-in exceptions are not enough to describe a specific problem in our program.

To create a custom exception, we define a class that inherits from the built-in Exception class.

Syntax:

```
class CustomError(Exception):
    pass
```

Example:

```
class AgeError(Exception):
    pass

try:
    age = int(input("Enter your age: "))
    if age < 18:
        raise AgeError("You must be 18 or older.")
    print("Access granted.")
except AgeError as e:
    print(e)
```

Explanation:

- A custom exception AgeError is created
- raise keyword is used to generate the exception
- It is handled using except

Why Use Custom Exceptions?

- Makes code more **readable and meaningful**

- Helps in handling **specific conditions**
- Improves **error clarity and debugging**

Important Points

- Multiple exceptions can be handled using multiple except blocks
- We can group exceptions in one except using parentheses
- Custom exceptions are created using classes
- raise is used to trigger an exception manually

Conclusion

Handling multiple exceptions helps manage different types of errors effectively, while custom exceptions allow developers to define their own error conditions. Together, they make programs more structured, clear, and robust.

6. Class and Object (OOP Concepts):

Q.1. Understanding the concepts of classes, objects, attributes, and methods in Python.

Ans: Python is an object-oriented programming language. It allows us to organize code using **classes** and **objects**. These concepts help in writing clean, reusable, and structured programs.

1. Class

A **class** is like a blueprint or template used to create objects. It defines the properties (attributes) and behaviors (methods) that the objects will have.

Syntax:

```
class ClassName:
  # attributes and methods
```

Example:

```
class Student:
  pass
```

2. Object

An **object** is an instance of a class. It represents a real-world entity and contains actual data.

Example:

```
s1 = Student()
s2 = Student()
```

Here, s1 and s2 are objects of the Student class.

3. Attributes

Attributes are variables that belong to a class or object. They store data related to the object.

Example:

```
class Student:
  def __init__(self, name, age):
    self.name = name
    self.age = age

s1 = Student("Mithlesh", 25)
print(s1.name)
print(s1.age)
```

Explanation:

- name and age are attributes
- self refers to the current object

4. Methods

Methods are functions defined inside a class. They describe the behavior of an object.

Example:

```
class Student:
  def __init__(self, name):
    self.name = name

  def greet(self):
    print("Hello, my name is", self.name)

s1 = Student("Mithlesh")
s1.greet()
```

Explanation:

- greet() is a method
- It uses object data (self.name)

Complete Example

```
class Car:
  def __init__(self, brand, model):
    self.brand = brand
    self.model = model

  def display(self):
    print("Car:", self.brand, self.model)

car1 = Car("Toyota", "Fortuner")
car1.display()
```

Key Points

- **Class** → Blueprint
- **Object** → Instance of class
- **Attributes** → Variables (data)
- **Methods** → Functions (behavior)
- self → Refers to the current object

Conclusion

Classes and objects are the foundation of object-oriented programming in Python. Attributes store data, and methods define actions. Using these concepts, we can model real-world problems efficiently and build structured programs.

Q.2. Difference between local and global variables.

Ans: In Python, variables can be defined in different parts of a program. Based on where they are created, they are mainly classified into **local variables** and **global variables**.

---

1. Local Variables

A **local variable** is a variable that is declared inside a function. It can only be used within that function and is not accessible outside.

Example:

```
def show():
  x = 10   # local variable
  print("Value of x:", x)

show()
```

Explanation:

- x is defined inside the function show()
- It can only be accessed within that function
- Trying to use x outside will cause an error

---

2. Global Variables

A **global variable** is a variable that is declared outside all functions. It can be accessed from anywhere in the program.

Example:

```
x = 20   # global variable

def show():
   print("Value of x:", x)

show()
print(x)
```

Explanation:

- x is defined outside the function
- It can be used both inside and outside the function

Modifying Global Variables

To change a global variable inside a function, we use the global keyword.

Example:

```
x = 10

def change():
   global x
   x = 50

change()
print(x)
```

Key Differences

Feature	Local Variable	Global Variable
Definition	Inside a function	Outside all functions
Scope	Only within the function	Entire program
Lifetime	Exists during function call	Exists throughout program
Access	Not accessible outside	Accessible everywhere

Important Points

- Local variables have **limited scope**
- Global variables have **wider scope**
- Use global variables carefully to avoid confusion
- Prefer local variables for better code safety

Conclusion

Local and global variables differ mainly in their scope and accessibility. Local variables are limited to functions, while global variables can be accessed throughout the program. Understanding this difference helps in writing clear and efficient Python code.

7. Inheritance:

Q.1. Single, Multilevel, Multiple, Hierarchical, and Hybrid inheritance in Python.

Ans: Inheritance is an important concept in object-oriented programming. It allows one class (child class) to **use the properties and methods** of another class (parent class). This helps in code reuse and better organization.

---

1. Single Inheritance

In single inheritance, a child class inherits from one parent class.

Example:

```
class Parent:
  def show(self):
    print("This is parent class")

class Child(Parent):
  def display(self):
    print("This is child class")

obj = Child()
obj.show()
obj.display()
```

Explanation:

- Child inherits from Parent
- It can access methods of both classes

---

2. Multilevel Inheritance

In multilevel inheritance, a class inherits from another class, which itself inherits from another class.

Example:

```
class Grandparent:
  def show(self):
    print("This is grandparent class")

class Parent(Grandparent):
  def display(self):
    print("This is parent class")

class Child(Parent):
  def print_msg(self):
    print("This is child class")

obj = Child()
obj.show()
obj.display()
obj.print_msg()
```

Explanation:

- Child inherits from Parent
- Parent inherits from Grandparent
- So Child can access all methods

---

3. Multiple Inheritance

In multiple inheritance, a child class inherits from more than one parent class.

Example:

```
class Father:
    def skill1(self):
        print("Driving")

class Mother:
    def skill2(self):
        print("Cooking")

class Child(Father, Mother):
    def skills(self):
        print("Child skills")

obj = Child()
obj.skill1()
obj.skill2()
obj.skills()
```

Explanation:

- Child inherits from both Father and Mother
- It can access methods from both classes

---

4. Hierarchical Inheritance

In hierarchical inheritance, multiple child classes inherit from the same parent class.

Example:

```
class Parent:
    def show(self):
        print("This is parent class")

class Child1(Parent):
    def display1(self):
        print("Child 1")

class Child2(Parent):
    def display2(self):
        print("Child 2")
```

```
obj1 = Child1()
obj2 = Child2()

obj1.show()
obj2.show()
```

Explanation:

- Both Child1 and Child2 inherit from Parent
- They share common features

---

5. Hybrid Inheritance

Hybrid inheritance is a combination of two or more types of inheritance.

Example:

```
class A:
  def method_a(self):
    print("Class A")

class B(A):
  def method_b(self):
    print("Class B")

class C(A):
  def method_c(self):
    print("Class C")

class D(B, C):
  def method_d(self):
    print("Class D")

obj = D()
obj.method_a()
obj.method_b()
obj.method_c()
obj.method_d()
```

Explanation:

- Combines multilevel and multiple inheritance
- D inherits from both B and C

Important Points

- Inheritance promotes **code reuse**
- Helps in **maintaining and organizing code**
- Python supports all types of inheritance
- Multiple inheritance follows **Method Resolution Order (MRO)**

Conclusion

Inheritance is a powerful feature in Python that allows classes to reuse and extend existing functionality. Types like single, multilevel, multiple, hierarchical, and hybrid inheritance help in designing flexible and efficient programs.

Q.2. Using the super() function to access properties of the parent class.

Ans: In Python, the super() function is used to access methods and properties of a **parent class** from a **child class**. It is mainly used in inheritance to avoid rewriting code and to make programs more efficient.

Why Use super()?

- To **reuse parent class code**
- To call the **parent class constructor (__init__)**
- To avoid directly using the parent class name
- Makes code more **maintainable and clean**

Basic Syntax:

super().method_name(arguments)

Example 1: Accessing Parent Method

```
class Parent:
  def show(self):
    print("This is parent class method")

class Child(Parent):
  def display(self):
    super().show()
    print("This is child class method")

obj = Child()
obj.display()
```

Explanation:

- Child inherits from Parent
- super().show() calls the parent class method
- Both parent and child methods are executed

Example 2: Using super() with Constructor

```
class Parent:
  def __init__(self, name):
    self.name = name

class Child(Parent):
  def __init__(self, name, age):
    super().__init__(name)
    self.age = age

obj = Child("Mithlesh", 25)
print(obj.name)
print(obj.age)
```

Explanation:

- super().__init__(name) calls the parent constructor
- It initializes the name attribute
- Child class adds its own attribute age

Example 3: Without Using super() (for comparison)

```
class Child(Parent):
  def __init__(self, name, age):
    Parent.__init__(self, name)
    self.age = age
```

Note:

- This works, but using super() is better and recommended

Important Points

- super() refers to the parent class
- Commonly used in constructors (__init__)
- Helps avoid code duplication
- Useful in multiple inheritance with proper method resolution

---

Conclusion

The super() function is a powerful feature in Python that allows child classes to access and reuse parent class methods and properties. It improves code readability, reduces redundancy, and supports better object-oriented design.

8. Method Overloading and Overriding:

Q.1. Method overloading: defining multiple methods with the same name but different parameters.

Ans: Method overloading means defining **multiple methods with the same name but different parameters**. The method behaves differently based on the number or type of arguments passed.

Does Python Support Method Overloading?

Python does **not support traditional method overloading** like Java or C++.
If we define multiple methods with the same name, Python will only keep the **last defined method**.

Example (Not Working as Overloading):

```
class Demo:
  def add(self, a, b):
    return a + b

  def add(self, a, b, c):
    return a + b + c

obj = Demo()
print(obj.add(2, 3))   # Error
```

Explanation:

- The second add() method overrides the first one
- Python does not treat them as separate methods

How to Achieve Method Overloading in Python

Even though direct overloading is not supported, we can achieve similar behavior using:

---

1. Using Default Arguments

```
class Math:
  def add(self, a, b=0, c=0):
    return a + b + c

obj = Math()
print(obj.add(5))        # 5
print(obj.add(5, 10))    # 15
print(obj.add(5, 10, 15))  # 30
```

Explanation:

- Same method works with different numbers of arguments
- Default values allow flexibility

---

2. Using Variable-Length Arguments (*args)

```
class Math:
  def add(self, *args):
    return sum(args)

obj = Math()
print(obj.add(2, 3))       # 5
print(obj.add(2, 3, 4, 5))   # 14
```

Explanation:

- *args allows passing any number of arguments
- The method handles all cases dynamically

Key Points

- Python does not support true method overloading
- Last defined method overrides previous ones
- We use default arguments or *args to simulate overloading

Conclusion

Method overloading allows using the same method name with different parameters. Although Python does not support it directly, we can achieve similar functionality using flexible arguments. This makes Python simple and powerful at the same time.

Q.2. Method overriding: redefining a parent class method in the child class.

Ans: Method overriding means **redefining a method of the parent class in the child class**. When a child class has a method with the same name as the parent class, the child's method is executed instead of the parent's method.

Why Use Method Overriding?

- To **change or extend** the behavior of a parent class method

- To provide **specific implementation** in the child class
- Helps in achieving **runtime polymorphism**

Syntax:

```
class Parent:
  def method(self):
    # parent code

class Child(Parent):
  def method(self):
    # overridden code
```

Example:

```
class Animal:
  def sound(self):
    print("Animals make sound")

class Dog(Animal):
  def sound(self):
    print("Dog barks")

obj = Dog()
obj.sound()
```

Output:

Dog barks

Explanation:

- Dog class overrides the sound() method of Animal
- When called, the child class method runs instead of the parent method

---

Using super() with Method Overriding

We can still call the parent class method using super() if needed.

Example:

```
class Animal:
  def sound(self):
    print("Animals make sound")

class Dog(Animal):
  def sound(self):
    super().sound()
    print("Dog barks")

obj = Dog()
obj.sound()
```

Output:

```
Animals make sound
Dog barks
```

Key Points

- Method name must be the **same in both parent and child class**
- The child class method **overrides** the parent method
- super() can be used to call the parent method
- Supports **runtime polymorphism**

Conclusion

Method overriding allows a child class to modify or extend the behavior of a parent class method. It is useful for customizing functionality and is an important concept in object-oriented programming in Python.

9. SQLite3 and PyMySQL (Database Connectors):

Q.1. Introduction to SQLite3 and PyMySQL for database connectivity.

Ans: In Python, databases are used to store and manage data permanently. To connect Python programs with databases, we use database libraries like **SQLite3**

and **PyMySQL**. These libraries allow us to perform operations such as creating tables, inserting data, updating data, and retrieving data.

---

1. SQLite3 in Python

SQLite3 is a lightweight, file-based database system. It comes **built-in with Python**, so no installation is required.

Features:

- No separate server needed
- Data is stored in a single file
- Easy to use and fast
- Suitable for small to medium applications

Basic Example:

```
import sqlite3

# Connect to database (creates file if not exists)
conn = sqlite3.connect("mydatabase.db")

# Create cursor object
cursor = conn.cursor()

# Create table
cursor.execute("CREATE TABLE IF NOT EXISTS users (id INTEGER, name TEXT)")

# Insert data
cursor.execute("INSERT INTO users VALUES (1, 'Mithlesh')")

# Save changes
conn.commit()

# Fetch data
cursor.execute("SELECT * FROM users")
print(cursor.fetchall())
```

```
# Close connection
conn.close()
```

---

2. PyMySQL in Python

PyMySQL is a library used to connect Python with a **MySQL database**. Unlike SQLite, MySQL is a **server-based database system**.

Features:

- Requires MySQL server installation
- Suitable for large and scalable applications
- Supports multiple users and high performance

Installation:

```
pip install pymysql
```

Basic Example:

```
import pymysql

# Connect to MySQL database
conn = pymysql.connect(
   host="localhost",
   user="root",
   password="your_password",
   database="testdb"
)

cursor = conn.cursor()

# Create table
cursor.execute("CREATE TABLE IF NOT EXISTS users (id INT, name
VARCHAR(50))")

# Insert data
cursor.execute("INSERT INTO users VALUES (1, 'Mithlesh')")
```

```
# Save changes
conn.commit()

# Fetch data
cursor.execute("SELECT * FROM users")
print(cursor.fetchall())

# Close connection
conn.close()
```

Difference Between SQLite3 and PyMySQL

Feature	SQLite3	PyMySQL (MySQL)
Type	File-based database	Server-based database
Installation	Built-in with Python	Requires installation
Usage	Small applications	Large applications
Performance	Lightweight	More powerful and scalable

Important Points

- Use **SQLite3** for simple projects and local storage
- Use **PyMySQL** for real-world applications with large data
- Always use commit() to save changes
- Always close the connection after use

Conclusion

SQLite3 and PyMySQL are important tools for database connectivity in Python. SQLite3 is simple and built-in, while PyMySQL connects Python with MySQL for more advanced applications. Choosing the right database depends on the size and requirements of the project.

Q.2. Creating and executing SQL queries from Python using these connectors.

Ans: In Python, we can interact with databases by writing **SQL queries** and executing them using database connectors like **sqlite3** and **pymysql**. These queries help us to create tables, insert data, update records, delete data, and fetch information.

Steps to Execute SQL Queries in Python

1. Connect to the database
2. Create a cursor object
3. Write SQL query
4. Execute the query
5. Commit changes (if required)
6. Fetch results (for SELECT queries)
7. Close the connection

---

1. Using SQLite3

SQLite3 is simple and built into Python.

Example:

```
import sqlite3

# Step 1: Connect
conn = sqlite3.connect("mydb.db")

# Step 2: Create cursor
cursor = conn.cursor()

# Step 3 & 4: Execute SQL queries
cursor.execute("CREATE TABLE IF NOT EXISTS users (id INTEGER, name TEXT)")
cursor.execute("INSERT INTO users VALUES (1, 'Mithlesh')")

# Step 5: Commit changes
conn.commit()
```

```
# Step 6: Fetch data
cursor.execute("SELECT * FROM users")
rows = cursor.fetchall()

for row in rows:
    print(row)

# Step 7: Close connection
conn.close()
```

---

2. Using PyMySQL

PyMySQL is used to connect Python with a MySQL database.

Example:

```
import pymysql

# Step 1: Connect
conn = pymysql.connect(
    host="localhost",
    user="root",
    password="your_password",
    database="testdb"
)

# Step 2: Create cursor
cursor = conn.cursor()

# Step 3 & 4: Execute SQL queries
cursor.execute("CREATE TABLE IF NOT EXISTS users (id INT, name
VARCHAR(50))")
cursor.execute("INSERT INTO users VALUES (1, 'Mithlesh')")

# Step 5: Commit changes
conn.commit()

# Step 6: Fetch data
cursor.execute("SELECT * FROM users")
```

```
rows = cursor.fetchall()

for row in rows:
    print(row)

# Step 7: Close connection
conn.close()
```

Common SQL Operations

- **CREATE** → Create table
- **INSERT** → Add data
- **SELECT** → Retrieve data
- **UPDATE** → Modify data
- **DELETE** → Remove data

Using Parameters (Best Practice)

To avoid SQL injection, we should use parameterized queries.

Example (SQLite3):

```
cursor.execute("INSERT INTO users (id, name) VALUES (?, ?)", (2, "Rahul"))
```

Example (PyMySQL):

```
cursor.execute("INSERT INTO users (id, name) VALUES (%s, %s)", (2, "Rahul"))
```

Important Points

- Always use cursor.execute() to run SQL queries
- Use commit() for INSERT, UPDATE, DELETE
- Use fetchall() or fetchone() for SELECT
- Close connection after completing work
- Prefer parameterized queries for security

Conclusion

Python allows easy execution of SQL queries using connectors like SQLite3 and PyMySQL. By following proper steps and best practices, we can efficiently manage and interact with databases in our applications.

10. Search and Match Functions:

Q.1. Using re.search() and re.match() functions in Python's re module for pattern matching.

Ans: In Python, the **re module** is used for working with **regular expressions (regex)**. Regular expressions help us to search, match, and manipulate text based on specific patterns.

Two important functions used for pattern matching are **re.search()** and **re.match()**.

---

1. re.match() Function

The re.match() function checks for a match **only at the beginning of the string**.

Syntax:

re.match(pattern, string)

Example:

```
import re

text = "Python is easy"
result = re.match("Python", text)

if result:
  print("Match found")
else:
  print("No match")
```

Output:

Match found

Explanation:

- The pattern "Python" is at the **start** of the string
- So, re.match() returns a match

Another Example:

```
result = re.match("easy", text)
print(result)
```

Output:

None

Explanation:

- "easy" is not at the beginning
- So, re.match() does not find a match

---

2. re.search() Function

The re.search() function checks for a match **anywhere in the string**.

Syntax:

```
re.search(pattern, string)
```

Example:

```
import re

text = "Python is easy"
result = re.search("easy", text)

if result:
  print("Match found")
else:
  print("No match")
```

Output:

Match found

Explanation:

- The pattern "easy" is present in the string
- re.search() finds it anywhere in the text

Key Difference Between re.match() and re.search()

Feature	**re.match()**	**re.search()**
Matching area	Only at beginning	Anywhere in string
Use case	Check starting pattern	Find pattern anywhere

Important Points

- Both functions return a **match object** if found, otherwise None
- Use .group() to get matched text
- Import re module before using these functions

Example using .group():

```
result = re.search("Python", "I love Python")

if result:
  print(result.group())
```

Conclusion

The re.match() and re.search() functions are useful for pattern matching in Python. re.match() is used when we want to check the beginning of a string, while re.search() is used to find patterns anywhere in the string. Understanding these functions helps in efficient text processing and validation.

Q.2. Difference between search and match.

Ans: In Python, both **re.search()** and **re.match()** functions are used for pattern matching using the **re module** (regular expressions). However, they work differently based on where they look for the pattern in a string.

---

1. re.match()

- Checks for a match **only at the beginning of the string**
- If the pattern is not at the start, it returns **None**

Example:

```
import re

text = "Python is easy"

result = re.match("Python", text)
print(result)   # Match found

result = re.match("easy", text)
print(result)   # None
```

---

2. re.search()

- Checks for a match **anywhere in the string**
- Returns the first occurrence of the pattern

Example:

```
import re

text = "Python is easy"

result = re.search("easy", text)
print(result)   # Match found
```

Key Differences

Feature	re.match()	re.search()
Matching position	Only at the beginning	Anywhere in the string
Flexibility	Less flexible	More flexible
Return value	Match object or None	Match object or None

Simple Explanation

- Use **re.match()** when you want to check if a string **starts with a pattern**
- Use **re.search()** when you want to find a pattern **anywhere in the string**

Conclusion

Both re.match() and re.search() are useful for pattern matching, but they differ in where they search. Choosing the right function depends on whether you want to match the beginning of a string or search the entire string.